RELAY AI

THE AUTONOMOUS AI ENGINEERING TEAM

From a reported issue to an evidence-backed, tested code change —
with human approval at every critical boundary.

PROJECT PURPOSE A multi-agent engineering workflow that receives software issues through a team chat, investigates a connected repository, proposes a fix, runs tests, prepares a reviewable change, and reports its evidence. Human developers retain approval authority.

HACKATHON PROJECT PROPOSAL & IMPLEMENTATION BLUEPRINT

Prepared for a four-member team | 04 November 2026

Document Control

Item	Details
Project name	RELAY AI
Product category	Multi-agent AI workflow / developer productivity
Primary hackathon fit	Build for Builders; Best Business Workflow
Team size	Four members
Document status	Proposed solution and build plan
Version	1.0
Date	04 November 2026

IMPORTANT VALIDATION NOTE Competition names, dates, team limits, tracks, prize pool and judging weights in this document reflect the information discussed alongside the supplied screenshots and prior planning. Confirm the current official event pages, submission rules, deadlines, permitted tools and Switch integration requirements before relying on them. Do not treat an indicative schedule as an official organizer timetable.

Contents

- 1. Executive Summary
- 2. Hackathon Context and Strategy
- 3. Problem Statement and Opportunity
- 4. Product Vision, Goals and Scope
- 5. User Journey and End-to-End Workflow
- 6. Functional Requirements
- 7. Non-Functional Requirements and Safety
- 8. Proposed System Architecture
- 9. Technology Stack

- 10. Agent Responsibilities and Handoffs
- 11. Four-Member Team Structure
- 12. MVP Prioritization
- 13. Demonstration Plan
- 14. Delivery Roadmap
- 15. Testing and Evaluation Plan
- 16. Risks and Mitigations
- 17. Success Metrics
- 18. Submission and Presentation Strategy
- 19. Immediate Next Steps
- Appendix A. Suggested Repository Structure
- Appendix B. Example Issue and Agent Output

1. Executive Summary

RELAY AI is a collaborative AI engineering workspace designed to turn a software issue reported in a team communication channel into a structured, reviewable engineering outcome. A developer reports a bug; specialized agents classify it, inspect a connected codebase, propose a patch, execute relevant tests in an isolated environment, and prepare an evidence-backed review. A human developer decides whether the change is acceptable.

The product is designed around a practical distinction: agents must do useful work, not merely exchange messages. Each stage should produce an inspectable artifact — a task record, repository evidence, proposed diff, test report, review findings or pull request — with a traceable handoff.

CORE VALUE PROPOSITION RELAY AI reduces the coordination overhead between issue reporting, investigation, implementation and verification while keeping humans in control of code changes.

1.1 Product thesis

- A shared chat channel is the entry point for engineering work.
- A coordinator turns natural-language requests into explicit, trackable tasks.
- Specialized agents perform bounded work and pass structured outputs to one another.
- A sandbox and test suite provide evidence beyond the model's own claims.
- A human approval gate prevents unreviewed changes from reaching the main branch.

1.2 Recommended competition positioning

Competition	Recommended positioning	What to emphasize
Diwali Codefest 2026	Build for Builders	Developer productivity, practical impact, originality, working implementation and usability.
Flip the Switch Hackathon	Best Business Workflow	Switch as the collaboration and context layer; agents completing a real multi-step workflow inside a team environment.

The same core product can support both entries, but the demos should be tailored to each competition. The Flip the Switch submission must demonstrate real, central use of Switch rather than treating it as a decorative chat integration.

2. Hackathon Context and Strategy

The supplied competition screenshots show two separate events:

Event	Context captured	Strategic implication
BUILD / 04: Diwali Codefest 2026	Apex Institute of Technology, Chandigarh University. Team page shows a four-person team. Event title indicates an online qualifier and final.	Prioritize a complete, demonstrable prototype and clear problem-solution fit.
Flip the Switch Hackathon	DevTools Academy branding; event page describes connecting coding agents to Slack, Teams, Discord and Telegram through Switch. Screenshot shows a $10,000 prize pool and teams of up to four.	Switch must be central to the workflow. Demonstrate agents receiving context and delivering real work through a shared team channel.

2.1 Judging alignment (planning assumptions to verify)

Judging dimension discussed	How RELAY AI should respond
Impact and relevance	Show a realistic software issue and quantify workflow stages, time and outcomes.
Working implementation	Demonstrate a complete issue-to-tested-patch path, not disconnected mock screens.
Originality	Show agent specialization, structured handoffs, evidence and human approval.
Usability and presentation	Use a clear dashboard, understandable statuses, readable reports and a rehearsed demo.
Switch usage	Show Switch carrying shared context, agent coordination and status updates in a real team room.
Technical execution	Show repository inspection, isolated code execution, test output and controlled GitHub integration.

NO GUARANTEE No project concept guarantees a win. RELAY AI is a strategically aligned proposal, not a claim about competitors, judging outcomes or organizer endorsement.

3. Problem Statement and Opportunity

3.1 Problem statement

In small software teams, a bug report often triggers a fragmented sequence: someone clarifies the issue, another person searches the repository, a developer attempts a fix, tests are run, and a reviewer checks the change. Work is spread across chat messages, code hosting, terminals and individual memory. Context can be lost, responsibilities can be unclear, and progress is difficult to audit.

3.2 Target users

- Small software teams and early-stage startups.
- Student developer teams and hackathon teams.
- Maintainers of small-to-medium repositories.
- Technical leads who need visibility into issue resolution.

3.3 Current pain points

- Issue descriptions arrive in unstructured language.
- Investigations are repeated because context is not preserved.
- Developers spend time coordinating handoffs.

- AI-generated code may be presented without adequate test evidence.
- Teams need a clear approval boundary before code is merged.

3.4 Proposed opportunity

RELAY AI combines a team communication surface, an orchestrator, bounded specialist agents, repository tooling and an auditable approval process. It aims to make the work visible and reviewable rather than hiding it behind a single AI response.

4. Product Vision, Goals and Scope

4.1 Vision

Make an AI engineering team a reliable collaborator inside the team's existing workflow — with clear responsibility, evidence and human control.

4.2 Primary goals

- Accept a bug report through a supported team chat integration.
- Convert the report into a structured task with severity, context and acceptance criteria.
- Inspect a connected repository and cite relevant files or code locations in the investigation.
- Generate a proposed fix in an isolated branch or workspace.
- Run relevant automated tests in a restricted execution environment.
- Produce a review summary and request human approval before creating or merging a change.
- Maintain a timeline of agent actions, artifacts, failures and decisions.

4.3 In scope for the hackathon MVP

- One chat platform / Switch workflow.
- One GitHub repository.
- One supported programming language or a deliberately narrow project type.
- A small number of specialist agents orchestrated in a deterministic workflow.
- A single end-to-end issue scenario.
- Human approval before any externally visible code change.

4.4 Out of scope for the MVP

- Autonomous production deployment.
- Unrestricted access to arbitrary repositories or credentials.
- Supporting every programming language and build system.
- Multiple chat platforms simultaneously.
- Voice commands, complex analytics and broad project management.
- Automatic merge to the protected main branch.

5. User Journey and End-to-End Workflow

Stage	What happens
Report	A teammate posts a bug report in the configured team room.
Triage	Triage Agent extracts the issue, classifies severity, identifies missing information and creates a task.
Investigation	Investigator Agent searches the permitted repository and returns likely root cause with file-level evidence.
Implementation	Developer Agent proposes a minimal change in an isolated workspace or branch.
Verification	Test Agent runs the selected tests and captures exit codes, logs and results.
Review	Review Agent inspects the diff and test evidence for correctness, security and regression risks.
Approval	A human reviews the patch and explicitly approves or rejects the next action.
Resolution	The system creates a pull request or review artifact and posts a concise resolution report.

5.1 Example scenario

A demo e-commerce application calculates the checkout total incorrectly when a discount is applied. A teammate reports the issue. RELAY AI identifies the relevant calculation, proposes a correction, runs the relevant tests and presents the diff and test evidence for human review.

DEMO RULE Use a deliberately seeded, reproducible bug in a demo repository. Do not claim the system can reliably fix arbitrary production defects based on one successful demonstration.

6. Functional Requirements

ID	Requirement	Acceptance evidence	Priority
FR-01	Receive an issue from the configured chat room.	A message creates a task with a unique ID.	P0
FR-02	Normalize the issue into structured fields.	Title, description, severity, repository and acceptance criteria are visible.	P0
FR-03	Investigate a connected repository.	Agent returns relevant files and evidence-backed findings.	P0
FR-04	Generate a proposed code change.	A diff is produced in an isolated workspace or branch.	P0
FR-05	Run relevant tests.	Test command, exit status and output are recorded.	P0
FR-06	Review the proposed change.	Review report lists findings, risks and unresolved questions.	P0
FR-07	Require human approval.	No PR creation/merge or protected action occurs before approval.	P0
FR-08	Publish a resolution summary.	Chat receives status, links/artifacts and remaining risks.	P0
FR-09	Create a GitHub pull request.	A PR is created only after explicit approval.	P1
FR-10	Display task timeline and agent status.	Dashboard shows stage, owner, timestamps and artifacts.	P1
FR-11	Support multiple repositories and chat platforms.	Configuration supports additional integrations.	P2

6.1 Minimum task record

task_id
reported_by
source_channel
repository
issue_title
issue_description
severity
acceptance_criteria
current_stage
assigned_agent
created_at / updated_at
investigation_artifact
proposed_diff
test_report
review_report
human_decision
final_status

7. Non-Functional Requirements and Safety

Area	Requirement
Security	Use least-privilege GitHub tokens; never expose secrets in prompts, logs or chat.
Isolation	Run repository commands and tests in a disposable, restricted container.
Human control	Require explicit approval for PR creation and any merge or deployment action.
Traceability	Record agent identity, input reference, output, timestamps and action results.
Reliability	Handle timeouts, malformed model output, tool errors and test failures without silently advancing.
Transparency	Distinguish verified results from model hypotheses; show evidence and uncertainty.
Performance	Keep the demo workflow responsive; show progress and meaningful intermediate status.
Privacy	Only process repositories and chat data the team is authorized to access.

SECURITY HARD STOP Do not execute arbitrary model-generated shell commands on a developer's host machine. Use an isolated container with restricted network access, resource limits, read-only inputs where possible, and a disposable workspace.

8. Proposed System Architecture

The architecture separates the communication layer, orchestration layer, specialist agents, tool adapters and evidence store. Switch should provide the shared collaboration context for the Flip the Switch entry.

```
TEAM MEMBER
|
v
CHAT PLATFORM / SWITCH SHARED ROOM
|
v
CHAT CONNECTOR + AUTHENTICATION
|
v
RELAY AI ORCHESTRATOR
|---- Task state and workflow controller
|---- Context / event store
|---- Agent handoff and retry policy
|
+---- TRIAGE AGENT
+---- INVESTIGATOR AGENT ---- GitHub repository reader
+---- DEVELOPER AGENT ------ Isolated workspace / patch tool
+---- TEST AGENT ---------- Restricted Docker runner
+---- REVIEW AGENT --------- Diff + test evidence reviewer
|
v
HUMAN APPROVAL GATE
|
v
GITHUB PULL REQUEST / REVIEW ARTIFACT
|
v
CHAT RESOLUTION REPORT + AUDIT TIMELINE
```

8.1 Architecture components

Component	Responsibility
Chat connector	Receives messages and posts status updates through the supported chat/Switch interface.
Orchestrator	Owns task state, controls the sequence, validates outputs and manages retries.
Agent layer	Performs specialized tasks with constrained tools and structured output schemas.
Repository adapter	Reads permitted files, branches and metadata from GitHub.
Execution sandbox	Runs approved test commands in a disposable environment.
Evidence store	Persists task state, artifacts, logs and approval decisions.
Dashboard	Displays workflow status, agent activity, diff, test results and approval controls.

8.2 Suggested workflow states

RECEIVED -> TRIAGED -> INVESTIGATING -> FIX_PROPOSED -> TESTING -> REVIEW_READY -> AWAITING_APPROVAL -> APPROVED -> PR_CREATED -> RESOLVED

Alternative terminal states: NEEDS_INFORMATION | TEST_FAILED | REJECTED | BLOCKED | ERROR

8.3 Switch integration principle

For Flip the Switch, demonstrate the actual supported Switch workflow and its agent/tool connections. Confirm the official SDK, API, setup instructions, supported chat providers and event model from the event's current documentation. Avoid inventing endpoints or assuming a particular integration method. The core experience should show a team member invoking the workflow, agents using shared context and the result returning to the same team space.

9. Recommended Technology Stack

Layer	Recommended option	Reason / implementation note
Frontend	React + Tailwind CSS	Fast dashboard development and reusable components.
Backend	Python + FastAPI	Straightforward API layer and Python ecosystem.
Agent orchestration	LangGraph or lightweight custom state machine	Use LangGraph if the team knows it; otherwise a small explicit state machine is easier to debug under time pressure.
LLM provider	Gemini API or OpenAI API	Choose one primary model provider first; keep the provider behind an adapter.
Communication	Switch + one supported chat provider	Switch is central to the second hackathon proposition.
Repository integration	GitHub API	Read repository context and create a PR after approval.
Execution	Docker sandbox	Isolate tests and generated code execution.
Database	SQLite for MVP	Simple local persistence; migrate only if required.
Version control	Git + GitHub	Team collaboration and reviewable changes.

STACK DISCIPLINE Do not integrate several LLM providers, frameworks or chat platforms at once. Select one working path and finish the end-to-end workflow before adding optional integrations.

10. Agent Responsibilities and Handoffs

Agent	Primary responsibility	Required output
Triage Agent	Understand and classify the reported issue; ask for missing information when necessary.	Structured issue object: title, severity, reproduction details, acceptance criteria and confidence.
Investigator Agent	Inspect permitted repository files and identify likely root cause.	Findings with file paths, relevant snippets/locations, reasoning and uncertainty.
Developer Agent	Prepare the smallest reasonable candidate fix.	Patch/diff, files changed, implementation rationale and known limitations.
Test Agent	Run selected tests in the sandbox.	Commands executed, exit codes, logs, passed/failed tests and environment details.
Review Agent	Review the proposed diff against requirements and evidence.	Findings by severity, regression/security concerns and recommendation.
Orchestrator	Coordinate agents, validate schemas and control state transitions.	Task timeline, retries, status and audit events.

10.1 Agent contract

Each agent should receive only the context and tools needed for its stage. Each agent must return structured data, not only free-form prose. The orchestrator validates the output before advancing.

```
{
"task_id": "REL-001",
"agent": "investigator",
"status": "completed",
"summary": "Likely discount calculation defect",
"evidence": [
{"file": "src/checkout.py", "location": "calculate_total", "reason": "Discount applied twice"}
],
"confidence": 0.82,
"uncertainties": ["No integration test for stacked discounts"],
"recommended_next_step": "developer"
}
```


11. Four-Member Team Structure

Member	Role	Ownership	Deliverables
Member 1	Backend & Architecture / Technical Lead	FastAPI, orchestrator, task states, model API adapter, database and system integration.	Running backend, workflow controller, task API and integration contract.
Member 2	AI & Agent Development	Triage, investigation, developer and review agent logic; prompts and output schemas.	Working specialist agents, structured outputs and agent handoffs.
Member 3	Frontend & Integration	React dashboard, chat/Switch integration, GitHub integration and user experience.	Dashboard, team-room interaction and artifact display.
Member 4	Testing, DevOps & Presentation	Docker, test harness, end-to-end QA, deployment, README, architecture visuals and demo.	Reproducible test environment, test evidence, documentation and presentation.

Member 4 should contribute to integration and test automation throughout the build. Documentation and presentation are important, but should not become an isolated full-time assignment.

11.1 Team operating rules

- Agree on API contracts and data schemas before parallel implementation.
- Use one shared repository and short-lived feature branches.
- Integrate at least twice per day; do not wait until the final hours.
- Every feature must have an owner and a testable definition of done.
- Keep one person responsible for the final integrated build.
- Freeze scope when the end-to-end demo becomes reliable.

12. MVP Prioritization

Priority	Feature	Decision
P0	Chat-based issue reporting	Build first.
P0	AI issue classification	Build first.
P0	Repository investigation	Build first.
P0	Code-fix proposal	Build first, constrained to one demo repository.
P0	Automated testing	Build first; capture actual command and result.
P0	Human approval	Mandatory safety and trust feature.
P1	GitHub PR creation	Add if core workflow is stable; approval required.
P1	Multi-project dashboard	Only if time remains.
P2	Voice commands	Do not build for MVP.
P2	Multiple chat platforms	Do not build for MVP.
P2	Fully autonomous deployment	Explicitly out of scope.

MVP DEFINITION OF DONE A user can submit one seeded bug through the configured team channel; RELAY AI investigates the repository, produces a patch, runs tests, shows evidence and waits for a human decision. The workflow is repeatable from a clean setup.

13. Demonstration Plan

13.1 Recommended live demo

- Show the seeded checkout bug and its expected behavior.
- Post a concise bug report in the configured Switch-connected team room.
- Show the Triage Agent creating a structured task and assigning severity.
- Show the Investigator Agent identifying the relevant function and explaining the suspected defect with file evidence.
- Show the Developer Agent preparing a minimal patch in the isolated workspace.
- Show the Test Agent running the relevant test suite and returning real output.
- Show the Review Agent evaluating the diff and test evidence.
- Show the human approval screen and explicitly approve or reject the next action.
- Show the resulting pull request or review artifact and the final chat summary.

13.2 Suggested demo narration

“A teammate reports a checkout defect in the team room. Instead of asking the team to manually coordinate triage, investigation, implementation and testing, RELAY AI creates a traceable workflow. Each agent has a specific responsibility and must return evidence. The system proposes a patch and verifies it in an isolated environment. It does not silently merge code: a human reviews the diff and decides what happens next. The outcome is a reviewable engineering artifact, not just an AI answer.”

13.3 What judges should see on screen

- Task status and current agent.
- Agent handoff timeline.
- Repository evidence and relevant file locations.

- Actual code diff.
- Test command, exit code and result.
- Review findings and uncertainty.
- Human approval control.
- Final PR or resolution artifact.

14. Delivery Roadmap

The following is an aggressive working plan based on the dates discussed for the events. It is not an official event schedule. Confirm organizer deadlines and adjust the time blocks accordingly.

Date / phase	Focus	Expected output
Nov 4 — First build block	Freeze scope, create repository, assign roles, agree schemas and initialize backend/frontend.	Shared repo, working skeleton, task contract and team ownership.
Nov 4 — Core workflow	Connect chat/Switch, implement triage and repository investigation, create task state.	Issue enters system and reaches investigation stage.
Nov 4 — Working prototype	Generate patch, run one test command in Docker, show status in UI.	One complete seeded issue workflow.
Nov 4 — Submission preparation	End-to-end test, fix blockers, record backup demo, prepare README and submission.	Stable qualifier-ready prototype and submission assets.
Nov 5–6 — Refinement	Improve reliability, error handling, GitHub PR flow, UI and Switch demonstration.	Polished product and repeatable demo.
Nov 7 — Final readiness	Code freeze, verify links and permissions, rehearse presentation and submit before official deadline.	Final build, presentation, demo recording and complete submission.

TIME MANAGEMENT The previous planning discussion suggested an evening qualifier submission on November 4. Treat that time as unverified until checked against the official portal. If the deadline is earlier, cut optional features immediately and preserve a working demo.

15. Testing and Evaluation Plan

Test category	Test case	Pass condition
Functional	Submit valid bug report.	Task is created with required fields.
Functional	Submit incomplete report.	System requests missing details or marks task as blocked; it does not invent facts.
Agent handoff	Investigator completes its task.	Structured evidence is passed to the developer stage.
Repository	Repository is unavailable or permission denied.	Workflow stops safely and reports the failure.
Patch	Developer agent proposes a change.	Diff is captured in isolated workspace; protected branch remains unchanged.
Execution	Test command passes.	Actual exit code and output are stored.
Execution	Test command fails or times out.	Workflow marks failure/blocked and does not claim success.
Review	Patch contains a risky change.	Review agent flags it; human approval remains required.
Approval	Human rejects proposed change.	No PR creation or merge occurs.
Integration	Switch/chat event is duplicated.	Idempotency prevents duplicate task creation where feasible.
Security	Prompt includes malicious instructions or secret-like data.	Tool permissions remain constrained; secrets are not exposed.

15.1 Evaluation measures

- Task completion rate across repeated seeded scenarios.
- Time from issue report to review-ready patch.
- Percentage of proposed patches passing the selected test suite.
- Number of unsupported or unverified claims in agent reports.
- Human review acceptance/rejection rate.
- Failure recovery rate and clarity of error reporting.

16. Risks and Mitigations

Risk	Impact	Mitigation
Integration failure	High	Agree on contracts early; integrate continuously; maintain a known-good branch.
Agents produce inconsistent outputs	High	Use JSON schemas, validation, bounded tools and explicit state transitions.
Generated code is incorrect	High	Run tests, inspect diff, use review agent and require human approval.
Unsafe command execution	Critical	Use disposable restricted containers; no host shell execution from model output.
Switch integration takes too long	High	Build a minimal verified integration first; keep a fallback demo recording.
Scope expansion	High	Enforce P0/P1/P2 list; defer voice, multiple platforms and deployment.
LLM/API quota or outage	Medium	Use one primary provider, handle errors, cache safe demo fixtures and keep a fallback path.
Demo environment failure	High	Prepare a local reproducible environment and recorded backup demo.
Weak judging narrative	Medium	Lead with the problem, show the workflow, then prove outcomes with artifacts.

16.1 Brutally honest feasibility assessment

Dimension	Assessment	Reason
Problem relevance	Strong	Developer workflow friction is concrete and easy to explain.
Fit across both events	Strong, with different emphasis	Build for Builders at one event; Switch-powered business workflow at the other.
Technical complexity	High	Repository access, code execution, agent orchestration and chat integration are non-trivial.
Four-person feasibility	Good if tightly scoped	A single repository, one language and one seeded bug keep the project manageable.
Demo potential	Strong	A visible issue-to-tested-patch sequence is easy for judges to understand.
Main failure risk	High integration risk	Parallel modules can fail to connect; continuous integration is essential.

The most likely reason this project underperforms is not a lack of features. It is an incomplete end-to-end integration. A small workflow that demonstrably works is more persuasive than a broad architecture that exists only in slides.

17. Success Metrics

Metric	How to measure	MVP target
End-to-end completion	Successful complete seeded workflows / attempts	At least 3 repeatable runs before final demo.
Evidence coverage	Agent claims linked to files, logs or test outputs	Every key technical claim has supporting evidence or is labelled as a hypothesis.
Test reporting	Runs with captured command, exit code and output	100% of executed test runs recorded.
Approval enforcement	Protected actions without approval	Zero.
Demo reliability	Successful rehearsals / total rehearsals	At least 3 consecutive successful rehearsals.
Task visibility	Stages and handoffs visible in timeline	All P0 stages visible.

These are proposed internal targets for the team, not organizer-defined scoring thresholds.

18. Submission and Presentation Strategy

18.1 Diwali Codefest narrative

- Track: Build for Builders.
- Lead with developer coordination overhead and the need for reliable issue resolution.
- Show working implementation, practical impact, originality and usable interface.
- Include architecture, setup steps, limitations, test evidence and a short demo.

18.2 Flip the Switch narrative

- Track: Best Business Workflow, subject to current official track rules.
- Make Switch central: demonstrate the real team-room interaction and shared context.
- Show the agents doing work and handing artifacts across stages.
- Explain the workflow's value to a small software business: less coordination overhead and more traceable issue handling.
- Document exactly how Switch is configured and which supported tools/connectors are used.

18.3 Suggested 5-minute pitch structure

Time	Content
0:00–0:30	Hook: show the cost of a bug report getting lost across chat, code and testing.
0:30–1:00	Problem and target user.
1:00–1:30	RELAY AI solution and architecture in one slide.
1:30–3:45	Live demo: report, investigate, patch, test, review and approve.
3:45–4:30	Evidence, safeguards, impact metrics and Switch integration.
4:30–5:00	Closing value proposition and next steps.

18.4 Submission checklist

- Project title, concise description and target user are consistent across portal, README and pitch.
- Working demo URL or local run instructions are tested on a clean machine.

- Repository is accessible to judges and contains no secrets or personal tokens.
- README includes prerequisites, environment variables, setup, run and test commands.
- Architecture diagram and workflow are included.
- Demo video is recorded and tested.
- Switch integration is real, documented and visible in the demo.
- Known limitations and safety boundaries are stated honestly.
- Submission is completed before the verified official deadline.

19. Immediate Next Steps

- Confirm the official rules, submission deadlines, judging criteria, permitted AI tools and team registration for both competitions.
- Assess the actual skill level of all four members in Python, frontend, LLM APIs/agents, GitHub and Docker.
- Choose one primary LLM provider and one chat/Switch integration path.
- Create a single GitHub repository and agree on the task schema and API contracts.
- Prepare a small demo repository with one deliberately seeded, reproducible checkout bug and a test that initially fails.
- Build the full issue-to-tested-patch workflow before adding optional dashboard or analytics features.
- Rehearse the demo on a clean environment and prepare a recorded fallback.

Appendix A. Suggested Repository Structure

```
relay-ai/
■■■ backend/
■ ■■■ app/
■ ■ ■■■ main.py
■ ■ ■■■ api/
■ ■ ■■■ orchestrator/
■ ■ ■■■ agents/
■ ■ ■ ■■■ triage.py
■ ■ ■ ■■■ investigator.py
■ ■ ■ ■■■ developer.py
■ ■ ■ ■■■ tester.py
■ ■ ■ ■■■ reviewer.py
■ ■ ■■■ integrations/
■ ■ ■ ■■■ switch_chat.py
■ ■ ■ ■■■ github.py
■ ■ ■■■ sandbox/
■ ■ ■■■ schemas/
■ ■ ■■■ storage/
■ ■■■ tests/
■ ■■■ requirements.txt
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ components/
■ ■ ■■■ pages/
■ ■ ■■■ services/
■ ■■■ package.json
■■■ demo-repo/
■ ■■■ src/
■ ■■■ tests/
■■■ docs/
■ ■■■ architecture.md
■ ■■■ workflow.md
■ ■■■ demo-script.md
■■■ docker/
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

Appendix B. Example Issue and Agent Output

B.1 Example issue

Field	Example
Title	Checkout total is incorrect after applying a discount
Description	The checkout total is lower than expected when the discount is applied. Reproduces on the demo store checkout page.
Expected behavior	Discount is applied once to the eligible subtotal and the final total matches the documented calculation.
Actual behavior	The discount appears to be deducted twice.
Severity	Medium — illustrative demo classification
Acceptance criteria	A regression test fails before the fix and passes after the fix; existing checkout tests remain green.

B.2 Example resolution report

RELAY AI — TASK REL-001
Status: AWAITING HUMAN APPROVAL

Issue: Checkout total is incorrect after discount
Investigation: Suspected duplicate discount application
Evidence: [repository file and function reference]
Proposed change: [diff artifact / branch]
Tests: [command executed]
Result: [actual pass/fail count and exit code]
Review: [findings, risks and unresolved questions]
Human decision: Pending

No merge or deployment has been performed.

Appendix C. Assumptions and Items to Verify

- The competition details and schedule must be confirmed against current official event pages.
- The supported Switch integration method and capabilities must be taken from official Switch documentation.
- The team has authorization to connect the selected GitHub repository and chat workspace.
- The demo repository is synthetic or explicitly approved for testing.
- The selected LLM API and development tools are available to the team under applicable terms and quotas.
- No claim of production-grade security, autonomous correctness or guaranteed hackathon success is made.